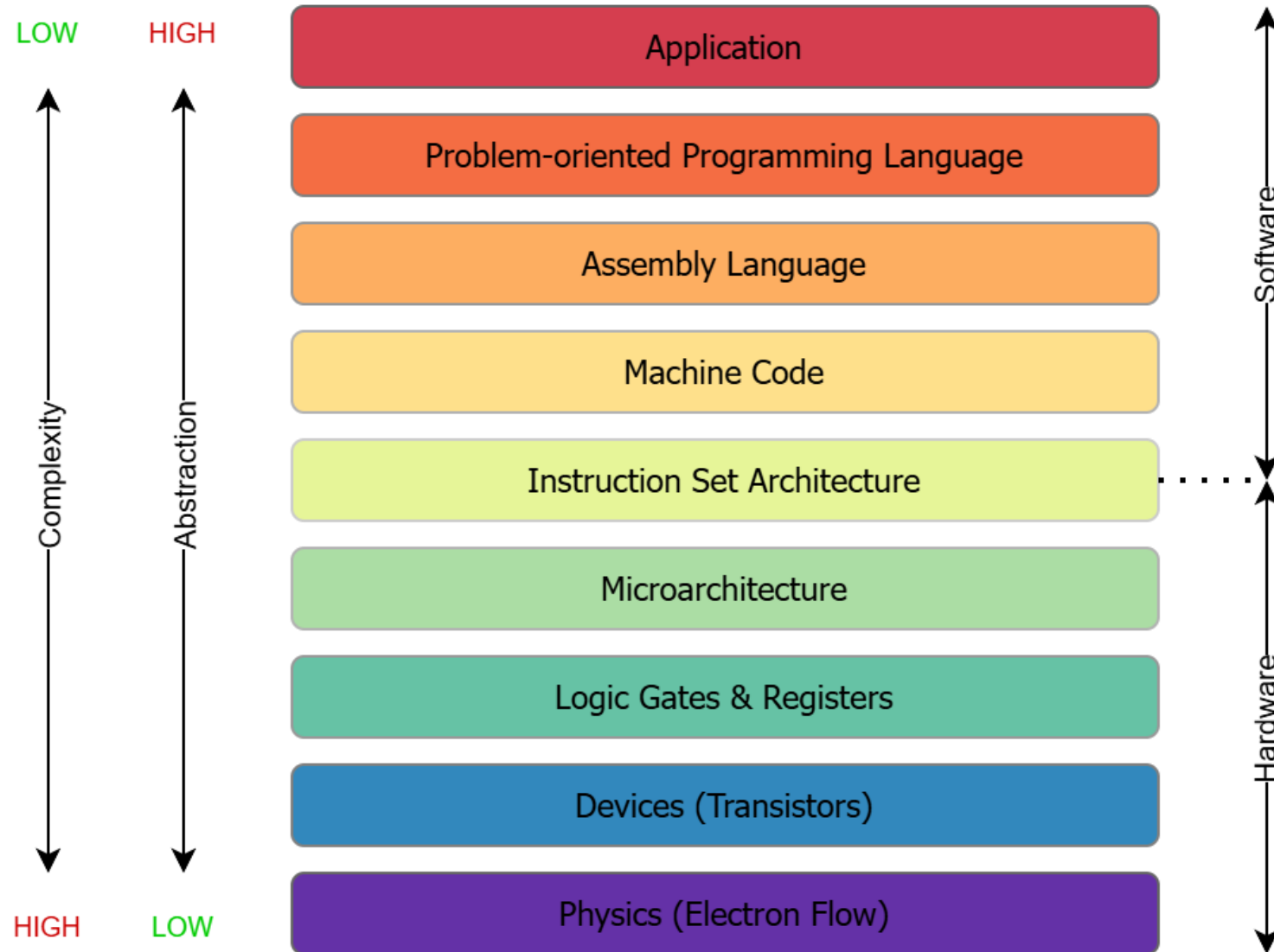


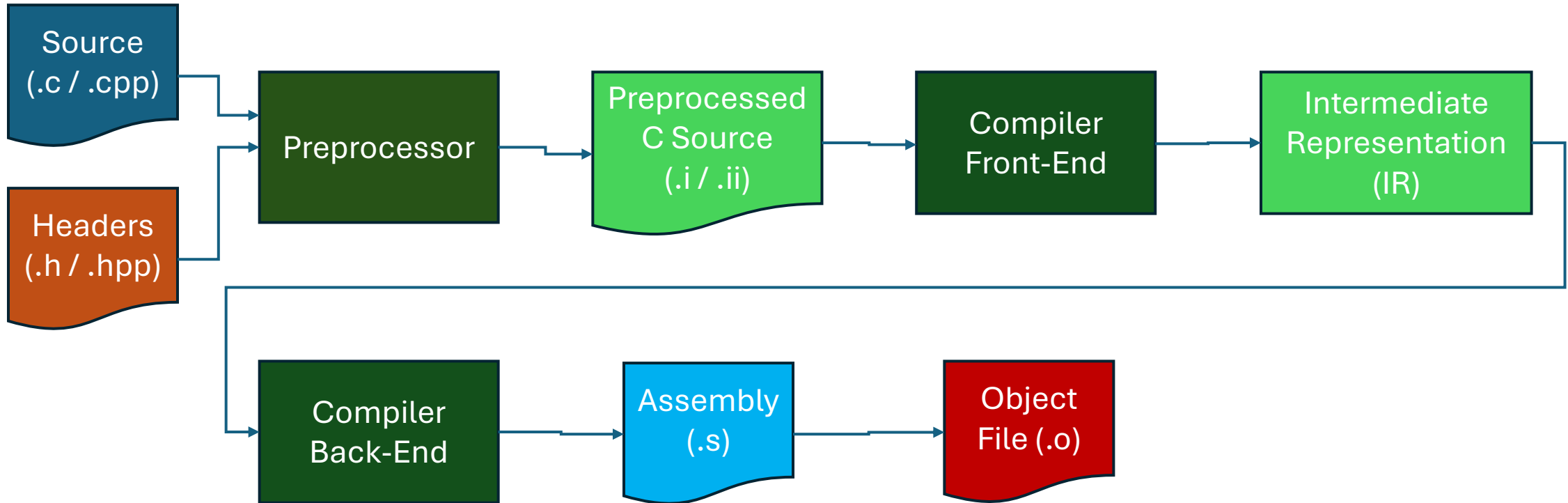
Computer Architecture: The Compiler, Linker, Loader

Dr. Colin O'Flynn

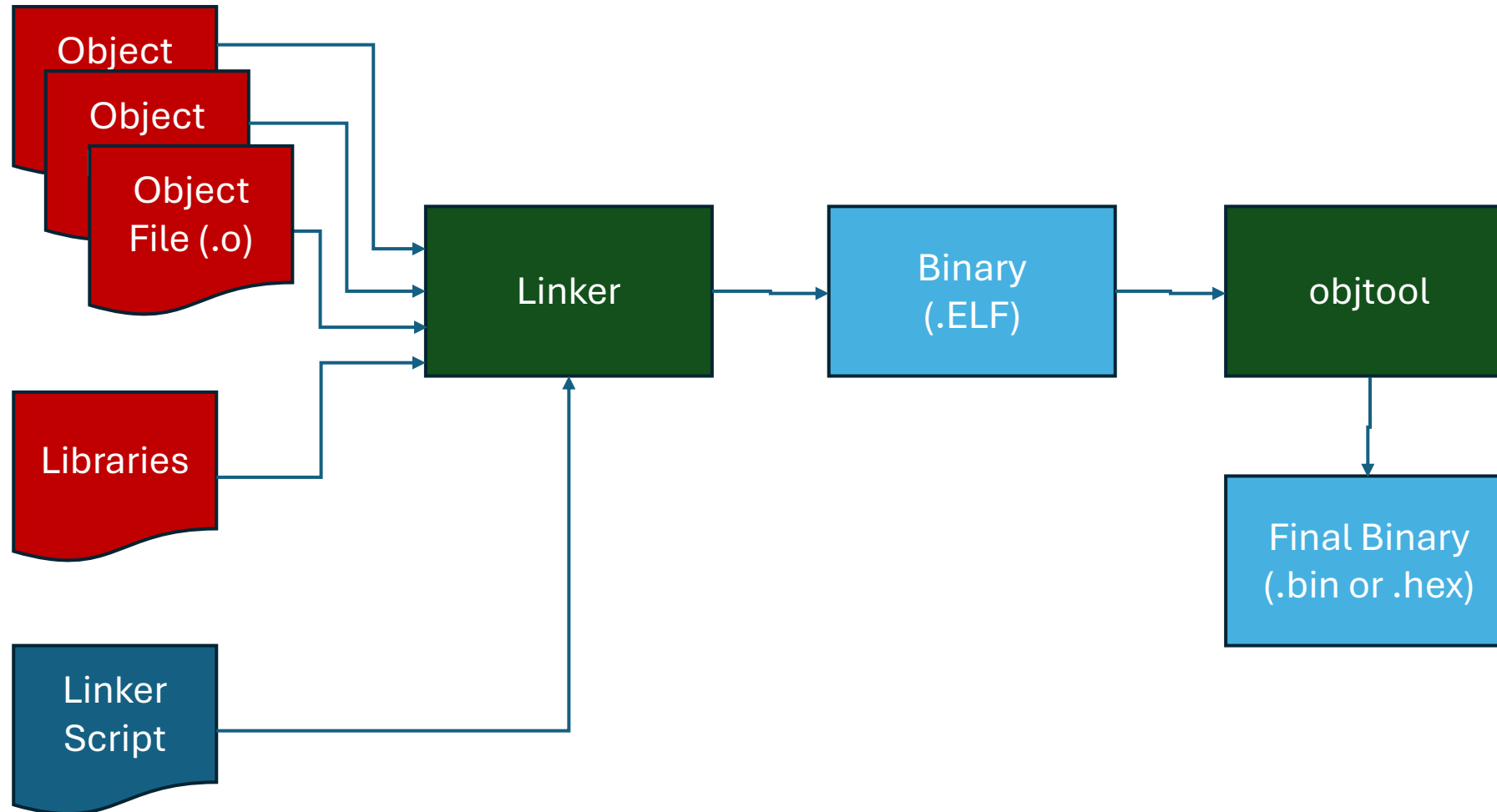
Layers of Abstraction



High Level Language → Running Code



High Level Language → Running Code



C/C++ Source Code (.c + .h)

```
1  #define GPIO_PIN 2
2  |
3  void gpio_high (int pin);
4  void gpio_low (int pin);
5
6
7  int main(void){
8
9      gpio_high(GPIO_PIN);
10
11 }
```

Preprocessed Source Code (.i / .ii)

```
1  /* <5 lines filtered> */
2
3  void gpio_high (int pin);
4  void gpio_low (int pin);
5
6
7  int main(void){
8      |
9      |    gpio_high(2);
10     |
11     |}
12
```

e.g., Preprocessor Dangers #1

```
#define SQUARE(x) x * x
```

...

```
int result = SQUARE(4 + 1);
```

Results of Compiler Explorer

```
1  main:
2      mov     r0, #9
3      bx      lr
```

```
1  /* <7 lines filtered> */
2
3  void gpio_high (int pin);
4  void gpio_low (int pin);
5
6
7  int main(void){
8
9      int result = 4 + 1 * 4 + 1;
10     return result;
11
12 }
13
```

Try it @ <https://godbolt.org/>

e.g., Preprocessor Dangers #2

```
#define CtoF(x) (x * 1.8) + 32
```

```
//This works fine:
```

```
printf("Temp %dC is %dF\n", c, CtoF(c));
```

Try It!

```
#include <stdio.h>
#define CtoF(x) (x * 1.8) + 32
#define NORMAL_TEMP_C (50)

// User manual says max temp is 1.5x normal
// temp, but they use Fahrenheit so we convert
// to be sure we exactly follow their calc.

int test_temp(int c){
    if (CtoF(c) > CtoF(NORMAL_TEMP_C)*1.5) {
        printf("WARNING: Temperature Exceeded\n");
    }
}
```

```
1  .LC0:
2      .ascii "WARNING: Temperature Exceeded\000"
3  test_temp(int):
4      push    {r4, lr}
5      ldr     r0, .L3
6      bl      puts
7  .L3:
8      .word   .LC0
```



```
1  /* <943 lines filtered> */
2
3  int test_temp(int c){
4      if ((c * 1.8) + 32 > ((50) * 1.8) + 32*1.5) {
5          printf("WARNING: Temperature Exceeded\n");
6      }
7  }
8
```

Compiler Front-End (Target Independent)

- Parses C/C++ (or other language) syntax
- Builds an “Abstract Syntax Tree” (AST)
- Performs:
 - Type checking
 - Scope resolution
 - Basic semantic check

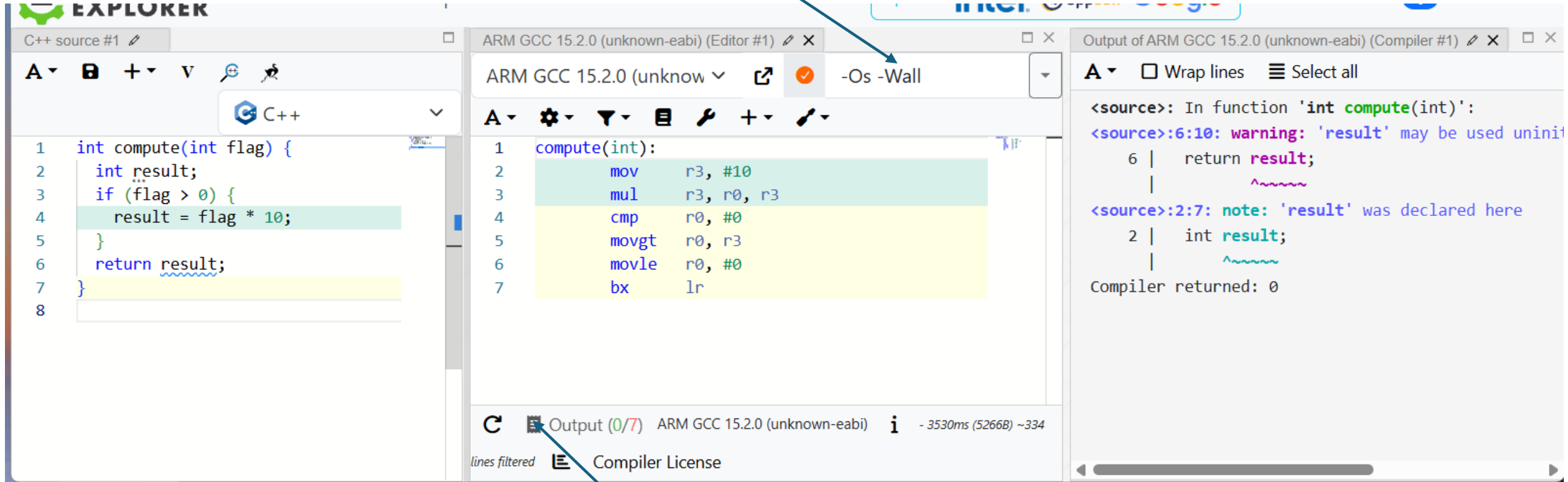
Basic Semantic Checks vs. Static Analysis

- Compiler provides basic checks on C/C++ correctness
- Can turn on various level of warnings to improve your detection

```
int compute(int flag) {  
    int result;  
    if (flag > 0) {  
        result = flag * 10;  
    }  
    return result;  
}
```

Try It!

Add “-Wall”



Click this button

Code Linting

Static Analysis does more intensive checking than simple compiler front ends.

cppcheck (for both C & C++) is a good tool!

<https://github.com/danmar/cppcheck>

Easy to install (Ubuntu):

```
sudo apt install cppcheck
```

Easy to use:

```
cppcheck bad.c (one file)
```

```
cppcheck folder (bunch of files)
```

Gimpel PC-Lint Bug Resources

Gimpel had PC-Lint, a great commercial linter.

Now owned by a bigger company, so you need archive.org for their more interesting history:

<https://web.archive.org/web/20190415235824/http://www.gimpel.com/html/bugs.htm>

<https://web.archive.org/web/20170627143350/http://www.gimpel.com/html/bugs/bug525.htm>

Classic Compiler Architecture

- Classic compiler architecture (e.g., early GCC) was *tightly coupled* to back-end
- Optimizations often *target-specific*



Low Level Virtual Machine (LLVM)

- Name is a reference to research on dynamic compilation – now encompasses many things
- Uses “Intermediate Representation” (IR)
- LLVM IR Optimizer optimizes generic IR code
- Originally designed to take IR output from GCC frontend... but...

Clang



- Originally developed by Apple for various reasons
 - Apple uses Objective-C (low priority for GCC devs)
 - GCC codebase is older & naturally larger being developed over many years
 - GCC is GPL licensed (potentially an issue commercially)
 - Desire to have a more modular tool that can be more easily integrated with other source-code tools
- Clang is more easily integrated/parsed
- Clang is now the LLVM frontend

New GCC vs. LLVM/Clang

- Internally GCC uses it's own Intermediate Representation
- Clang outputs LLVM IR
- GCC is a complete compiler to generate binary (object)
- Clang is not – requires LLVM to generate binary (object)

GCC vs. LLVM/Clang

- People have different opinions about GCC vs. Clang
- They have different output styles, warnings, may find more (or less) possible errors, etc.
- Historically Clang performance was worse than GCC (but faster to compile). This is no longer a generically true statement.

Intermediate Representation Example

- Basically – lower-level than C, higher-level than ASM
- Mostly “**Static Single Assignment**” (SSA)
- E.g.:

```
%1 = load a
%2 = load b
%3 = add %1, %2
store %3 -> c
```

Optimization

- SSA simplifies optimization
- Typically at minimum compiler will:
 - Remove unnecessary stack & memory load/stores
 - Dead code elimination (DCE)
- See <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

Optimization Examples

```
#include <stdio.h>

void gpio_on(void);
void gpio_off(void);
void gpio_toggle();

void toggle_led(void){
    //gpio_on();
    //gpio_off();
    gpio_toggle();
}

int some_function(int data){
    if (data > 0) {
        for(int i = 0; i < 4; i++){
            toggle_led();
        }
        return 0;
    }

    if (data < 0) {
        for(int i = 0; i < 10; i++){
            toggle_led();
        }
    }
    if (data > 0){
        toggle_led();
        return 2;
    }
    return 3;
}
```

```

1  #include <stdio.h>
2
3  void gpio_on(void);
4  void gpio_off(void);
5  void gpio_toggle();
6
7  void toggle_led(void){
8      ... //gpio_on();
9      ... //gpio_off();
10     ... gpio_toggle();
11 }
12
13 int some_function(int data){
14     ... if (data > 0) {
15         ... for(int i = 0; i < 4; i++){
16             ... toggle_led();
17         ... }
18         ... return 0;
19     ... }
20
21     ... if (data < 0) {
22         ... for(int i = 0; i < 10; i++){
23             ... toggle_led();
24         ... }
25     ... }
26     ... if (data > 0){
27         ... toggle_led();
28         ... return 2;
29     ... }
30     ... return 3;
31 }

```

```

ARM GCC 15.2.0 (unknown
-00

A Output... Filter... Libraries Overrides + Add new... Add tool...

1 toggle_led():
2     push    {fp, lr}
3     add     fp, sp, #4
4     bl      gpio_toggle()
5
6     nop
7     sub     sp, fp, #4
8     pop     {fp, lr}
9     bx      lr
10
11 some_function(int):
12     push    {fp, lr}
13     add     fp, sp, #4
14     sub     sp, sp, #16
15     str     r0, [fp, #-16]
16     ldr     r3, [fp, #-16]
17     cmp     r3, #0
18     ble     .L3
19     mov     r3, #0
20     str     r3, [fp, #-8]
21     b       .L4
22
23 .L5:
24     bl      toggle_led()
25     ldr     r3, [fp, #-8]
26     add     r3, r3, #1
27     str     r3, [fp, #-8]
28
29 .L4:
30     ldr     r3, [fp, #-8]
31     cmp     r3, #3
32     ble     .L5
33     mov     r3, #0
34     b       .L6
35
36 .L3:
37     ldr     r3, [fp, #-16]
38     cmp     r3, #0
39     bge     .L7
40     mov     r3, #0
41     str     r3, [fp, #-12]
42     b       .L8
43
44 .L9:
45     bl      toggle_led()
46     ldr     r3, [fp, #-12]
47     add     r3, r3, #1
48     str     r3, [fp, #-12]
49
50 .L8:
51     ldr     r3, [fp, #-12]
52     cmp     r3, #9
53     ble     .L9
54
55 .L7:
56     ldr     r3, [fp, #-16]
57     cmp     r3, #0
58     ble     .L10
59     bl      toggle_led()
60     mov     r3, #2
61     b       .L6
62
63 .L10:
64     mov     r3, #3
65
66 .L6:
67     mov     r0, r3
68     sub     sp, fp, #4
69     pop     {fp, lr}

```

```

1  #include <stdio.h>
2
3  void gpio_on(void);
4  void gpio_off(void);
5  void gpio_toggle();
6
7  void toggle_led(void){
8      //gpio_on();
9      //gpio_off();
10     gpio_toggle();
11 }
12
13 int some_function(int data){
14     if (data > 0) {
15         for(int i = 0; i < 4; i++){
16             toggle_led();
17         }
18         return 0;
19     }
20
21     if (data < 0) {
22         for(int i = 0; i < 10; i++){
23             toggle_led();
24         }
25     }
26     if (data > 0){
27         toggle_led();
28         return 2;
29     }
30     return 3;
31 }

```

```

1  toggle_led():
2      push    {r4, lr}
3      bl      gpio_toggle()
4      pop     {r4, lr}
5      bx      lr
6  some_function(int):
7      cmp     r0, #0
8      push    {r4, lr}
9      ble     .L4
10     bl      gpio_toggle()
11     bl      gpio_toggle()
12     bl      gpio_toggle()
13     bl      gpio_toggle()
14     mov     r0, #0
15 .L3:
16     pop     {r4, lr}
17     bx      lr
18 .L4:
19     beq     .L6
20     mov     r4, #10
21 .L7:
22     bl      gpio_toggle()
23     subs    r4, r4, #1
24     bne     .L7
25 .L6:
26     mov     r0, #3
27     b       .L3

```

```

1  #include <stdio.h>
2
3  void gpio_on(void);
4  void gpio_off(void);
5  void gpio_toggle();
6
7  void toggle_led(void){
8      //gpio_on();
9      //gpio_off();
10     gpio_toggle();
11 }
12
13 int some_function(int data){
14     if (data > 0) {
15         for(int i = 0; i < 4; i++){
16             toggle_led();
17         }
18         return 0;
19     }
20
21     if (data < 0) {
22         for(int i = 0; i < 10; i++){
23             toggle_led();
24         }
25     }
26     if (data > 0){
27         toggle_led();
28         return 2;
29     }
30     return 3;
31 }

```

Function inlining

Loop unrolling

Unreachable Code

```

1  toggle_led():
2      push    {r4, lr}
3      bl      gpio_toggle()
4      pop     {r4, lr}
5      bx      lr
6
7  some_function(int):
8      cmp     r0, #0
9      ble     .L4
10     bl      gpio_toggle()
11     bl      gpio_toggle()
12     bl      gpio_toggle()
13     bl      gpio_toggle()
14     mov     r0, #0
15
16 .L3:
17     pop     {r4, lr}
18     bx      lr
19
20 .L4:
21     beq     .L6
22     mov     r4, #10
23
24 .L7:
25     bl      gpio_toggle()
26     subs    r4, r4, #1
27     bne     .L7
28
29 .L6:
30     mov     r0, #3
31     b       .L3

```

Assembler Output

- After optimization the compiler writes an assembler file which is machine-dependant
- We looked at different assembly types in the last lecture
- Remember things are still “human readable” at this stage – note the use of labels, function names, etc

Object File

- Object file contains:
 - Machine code (no longer assembly)
 - Symbol table (function names, variables, etc)
 - Relocation entries
- Addresses & Object Files
 - Addresses are **not final** in the object file yet
 - Addresses of certain variables, functions etc are not known yet
 - The object file(s) will be finalized by the linker

Libraries

- We may use libraries which contain pre-compiled object files
- Normally this is for convenience (well optimized routines)
- Sometimes these may be commercial and provided in binary form
- Normally calling one function from a library may link in the entire library unless special work done to remove unused library code

Embedded Systems & Code Size

- If you have limited space, need to be very careful calling certain functions such as `printf()`

Embedded Standard Libraries

- Normal to have “C standard library” linked for things in `<stdio.h>`, `<strings.h>`, etc.
- When compiling for your desktop with GCC, you would normally have **glibc** linked in
- On embedded systems this has space issues...

Embedded Standard Libraries

- **newlib** is common on embedded
- **newlib-nano** is an even smaller version, common on Cortex-M (the most likely default of any toolchain you use)
- newlib-nano often has no printf() support for floating point

The Linker

- Combines object files (user + libraries)
- Resolves symbols to final addresses
- Assigns final addresses
- Need to ensure it is linking **startup code** (sometimes this gets removed by accident when changing linker options!)

Linker Scripts

- The Linker Script defines where in memory things go, e.g.:

```
FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
```

```
RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
```

Let's look in more detail at one, example from

<https://github.com/newaetech/chipwhisperer/blob/develop/firmware/mcu/hal/stm32f3/LinkerScript.ld>

```
/* Memories definition */  
MEMORY  
{  
    RAM (xrw)      : ORIGIN = 0x20000000, LENGTH = 40K  
    ROM (rx)       : ORIGIN = 0x8000000,  LENGTH = 256K  
}
```

```

/* Sections */
SECTIONS
{
    /* The startup code into ROM memory */
    .isr_vector :
    {
        . = ALIGN(4);
        KEEP(*(.isr_vector)) /* Startup code */
        . = ALIGN(4);
    } >ROM

    /* The program code and other data into ROM memory */
    .text :
    {
        . = ALIGN(4);
        *(.text)           /* .text sections (code) */
        *(.text*)          /* .text* sections (code) */
        *(.glue_7)          /* glue arm to thumb code */
        *(.glue_7t)         /* glue thumb to arm code */
        *(.eh_frame)

        KEEP (*(.init))
        KEEP (*(.fini))

        . = ALIGN(4);
        _etext = .;         /* define a global symbols at end of code */
    } >ROM

```

GCC Linker Script Options

You can pass options to the linker in gcc to specify what libraries to include.

Weird errors complaining about “no _write()” may be linker error setup!

Often on embedded we see `--specs=nosys.specs` as an option passed to the linker.

Memory Segments

.text (code) Segment:

- Compiled machine code instructions and read-only data, such as string literals and global constants
- Typically marked as read-only and executable to prevent accidental modification during program execution

Memory Segments

.data Segment:

- Stores global and static variables that are explicitly initialized by the programmer.
- This is a writable area of memory, as these variables can change during runtime.

Memory Segments

.bss (Block Started by Symbol) Segment:

- Holds uninitialized global and static variables, or those initialized to zero.
- Storing them in the BSS segment makes the file smaller; this section of memory is all written to zero on startup

Memory Segments

Heap Segment:

- Used for dynamic memory allocation requested during program execution using functions like `malloc()`, `calloc()`
- May or may not be used

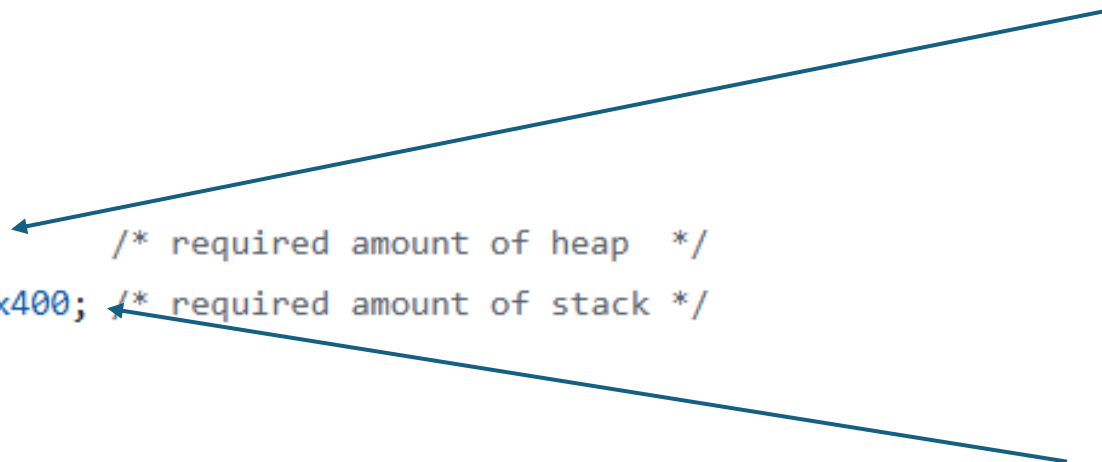
Memory Segments

Stack Segment:

- Used for managing function calls, local (automatic) variables, and return addresses
- Operates as a last-in, first-out (LIFO) structure
- Memory is automatically allocated and reclaimed as functions are called and return.
- Very bad things happen if you run out of this!

Linker Memory Allocation

```
57  
58  _Min_Heap_Size = 0;      /* required amount of heap */  
59  _Min_Stack_Size = 0x400; /* required amount of stack */  
60
```



Hint if Program is Crashing

- Lazy programmer trick: increase stack in the Linker Script and see if it works.
- If you do something with a lot of stack usage (allocating big variables on stack, using RTOS, etc) – may need to increase stack size!

Startup Code?

- Your code is NOT the first thing run, even on "bare metal"
- There is normally *startup code* that does some needed setup.

Following example is from STM32F3 default startup code, example at

https://github.com/newaetech/chipwhisperer/blob/develop/firmware/mcu/hal/stm32f3/stm32f3_startup.S

e.g., stm32f3 startup code

```
.section .text.Reset_Handler
        .weak      Reset_Handler
        .type      Reset_Handler, %function
Reset_Handler:

/* Copy the data segment initializers from flash to SRAM */
    movs    r1, #0
    b       LoopCopyDataInit

CopyDataInit:
    ldr     r3, =_sidata
    ldr     r3, [r3, r1]
    str     r3, [r0, r1]
    adds    r1, r1, #4

LoopCopyDataInit:
    ldr     r0, =_sdata
    ldr     r3, =_edata
    adds    r2, r0, r1
    cmp     r2, r3
    bcc     CopyDataInit
    ldr     r2, =_sbss
    b       LoopFillZerobss
```

```
/* Zero fill the bss segment. */
FillZerobss:
    movs    r3, #0
    str     r3, [r2]
    adds    r2, r2, #4

LoopFillZerobss:
    ldr     r3, =_ebss
    cmp     r2, r3
    bcc     FillZerobss

/* Call the clock system initialization function.*/
    bl     SystemInit
/* Call static constructors */
    bl     __libc_init_array
/* Call the application's entry point.*/
    bl     main

LoopForever:
    b       LoopForever
```

ELF Files and Debug Symbols

ELF (Executable and Linkable Format) is a standard file format for executable files & object code.

Supports extensive additional information (including debug information).

Normally this file then gets written to a final binary for the embedded system (.bin or .hex).

Debugging Optimized Code

- More difficult! Will see this once we get to debugging...

Loader?

- Normally we also have a loader that loads the executable and performs needed setup.
- This isn't normally relevant on small embedded systems – the entire binary is being run directly.
- More complex embedded systems (Linux) still have loaders.

Putting it Together – An Example

Running "Make"... part 1

```
arm-none-eabi-gcc (GNU Arm Embedded Toolchain 10-2020-q4-major) 10.2.1 20201103 (release)
Copyright (C) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

.
Compiling:
  simpleserial-aes.c ...Done!
.
Compiling:
  ../../simpleserial/simpleserial.c ...Done!
.
Compiling:
  ../../hal/stm32f3/stm32f3_hal.c ...Done!
.
Compiling:
  ../../hal/stm32f3/stm32f3_hal_lowlevel.c ...Done!
.
Compiling:
  ../../hal/stm32f3/stm32f3_sysmem.c ...Done!
.
Compiling:
  ../../crypto/tiny-AES128-C/aes.c ...Done!
.
Compiling:
  ../../crypto/aes-independant.c ...Done!
.
```

Make output continued

```
Assembling: ../../hal/stm32f3/stm32f3_startup.S
arm-none-eabi-gcc -c -mcpu=cortex-m4 -I. -x assembler-with-cpp -mthumb -mfloat-abi=soft -fmessage-length=0
ffunction-sections -DF_CPU=7372800 -Wa,-gstabs,-adhlns=objdir-CW308_STM32F3/stm32f3_startup.lst -I../../simp
leserial/ -I../../hal -I../../hal/stm32f3 -I../../hal/stm32f3/CMSIS -I../../hal/stm32f3/CMSIS/core -I../../hal/st
32f3/CMSIS/device -I../../hal/stm32f4/Legacy -I../../simpleserial/ -I../../crypto/ -I../../crypto/tiny-AES128-C
../../hal/stm32f3/stm32f3_startup.S -o objdir-CW308_STM32F3/stm32f3_startup.o
.
LINKING:
  simpleserial-aes-CW308_STM32F3.elf ...Done!
.
Creating load file for Flash: simpleserial-aes-CW308_STM32F3.hex
arm-none-eabi-objcopy -O ihex -R .eeprom -R .fuse -R .lock -R .signature simpleserial-aes-CW308_STM32F3.elf
simpleserial-aes-CW308_STM32F3.hex
.
Creating load file for Flash: simpleserial-aes-CW308_STM32F3.bin
arm-none-eabi-objcopy -O binary -R .eeprom -R .fuse -R .lock -R .signature simpleserial-aes-CW308_STM32F3.e
f simpleserial-aes-CW308_STM32F3.bin
.
Creating load file for EEPROM: simpleserial-aes-CW308_STM32F3.eep
arm-none-eabi-objcopy -j .eeprom --set-section-flags=.eeprom="alloc,load" \
--change-section-lma .eeprom=0 --no-change-warnings -O ihex simpleserial-aes-CW308_STM32F3.elf simpleserial
aes-CW308_STM32F3.eep || exit 0
```

Make output continued

```
Creating Extended Listing: simpleserial-aes-CW308_STM32F3.lss
arm-none-eabi-objdump -h -S -z simpleserial-aes-CW308_STM32F3.elf > simpleserial-aes-CW308_STM32F3.lss
.
Creating Symbol Table: simpleserial-aes-CW308_STM32F3.sym
arm-none-eabi-nm -n simpleserial-aes-CW308_STM32F3.elf > simpleserial-aes-CW308_STM32F3.sym
No CRYPTO_TARGET passed - defaulting to TINYAES128C
Building for platform CW308_STM32F3 with CRYPTO_TARGET=TINYAES128C
SS_VER set to SS_VER_1_1
SS_VER set to SS_VER_1_1
Blank crypto options, building for AES128
Size after:
  text    data    bss    dec    hex filename
  5712    536    1632    7880    1ec8 simpleserial-aes-CW308_STM32F3.elf
```

Additional Useful Outputs

 simpleserial-aes-CW308_STM32F3.bin	2026-01-14 2:07 PM	BIN File	7 KB
 simpleserial-aes-CW308_STM32F3.eep	2026-01-14 2:07 PM	EEP File	1 KB
 simpleserial-aes-CW308_STM32F3.elf	2026-01-14 2:07 PM	ELF File	190 KB
 simpleserial-aes-CW308_STM32F3.hex	2026-01-14 2:07 PM	HEX File	18 KB
 simpleserial-aes-CW308_STM32F3.lss	2026-01-14 2:07 PM	LSS File	129 KB
 simpleserial-aes-CW308_STM32F3.map	2026-01-14 2:07 PM	MAP File	57 KB
 simpleserial-aes-CW308_STM32F3.sym	2026-01-14 2:07 PM	SYM File	3 KB

.bin, .hex, and .eep are for programming (not useful to you)

.elf = for debugging (useful indirectly)

.lss = Listing Symbols, very useful to look at (may need to enable this)

.map = Memory map including symbols & source files

.sym = Symbols and location in memory

Live Example Pause Slide

This slide is to remind me live demos of:

- Building firmware & showing output
- Going through each of the prior files (lss, map, sym)
- Debugging example, show startup code

Summary

- Compiler converts our source code to some *intermediate representation (IR)*
- Optimization on this IR may change the code or program flow – **turning off optimizations should not fix your code**
- Machine-dependant assembly code is written – this is still missing startup code & final address assignments
- The linker performs final address assignment, and links in libraries (including startup code).